Total No. of Questions : 4]

SEAT No. :

P8465

[Total No. of Pages : 2

Oct-22/BE/Insem-41

B.E. (Computer Engineering)

DESIGN AND ANALYSIS OF ALGORITHMS

(2019 Pattern) (Semester - VII) (410241)

Time : 1 Hour] *[Max. Marks : 30*

Instructions to the candidates:

1) ***Answer the question of Q.1 or Q.2, Q.3 or Q.4.***
2) ***Neat diagrams must be drawn whenever necessary.***
3) ***Figures to the right indicate full marks.***
4) ***Assume suitable data, if necessary.***

Q1) a) Why correctness of the algorithm is important? Define loop invariant property and prove the correctness of finding summation of n numbers using loop invariant property. **[8]**

b) What is iterative algorithm? Explain iteractive algorithm design issues using examples. **[7]**

OR

Q2) a) How to prove that an algorithm is correct? How to prove the correctness of an algorithm using counter example? Give suitable example. **[7]**

b) Write a short note on any 4 problem solving strategies. **[8]**

Q3) a) What is Best, Average and Worst case Analysis of Algorithms? Analyse the following algorithm Best, Average and Worst case **[8]**

```
void sort (int a. int n) {
   int i, j;
   for (i = 0; i < n; i++) {
      j = i-1;
      key = a[i];
      while (j >=0 && a[j] > key)
      {
         a[j+1] = a[j];
         j = j-1;
      }
      a[j+1] = key;
   }
}
```

P.T.O.

CEGP013091 49.248.216.238 03/10/2022 13:41:09 static-238

b) • Explain P, NP, NP-Hard and NP-Complete problems with examples.

• Explain 3-SAT problem using an example. Why is SAT so important in theoretical computer science?

[7]

OR

Q4) a) What is NP-complete class problem? How would you prove vertex cover problem is NP-complete class problem? **[8]**

b) What is Best, Average and Worst case Analysis of Algorithms? Analyse the following algorithm Best, Average and Worst case **[7]**

```
int Linear-search(int a, int n, int item) {
   int i;
   for (i = 0; i < n; i++) {
      if (a[i] = = item) {
           return a[i]
      }
   }
   return - 1
```

⌘⌘⌘

CEGP013091 49.248.216.238 03/10/2022 13:41:09 static-238

Study material provided by: Vishwajeet Londhe

Join Community by clicking below links

Telegram Channel

https://t.me/SPPU_TE_BE_COMP

(for all engineering Resources)



WhatsApp Channel

(for all Engg & tech updates)

https://whatsapp.com/channel/0029ValjFriICVfpcV9HFc3b



Insta Page

(for all Engg & tech updates)

https://www.instagram.com/sppu_engineering_update

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.1

-> a)

(Need of Correctness of Algorithm)

Introduction:

- Algorithm correctness ensures that the algorithm produces the desired output for all valid inputs and behaves as expected in every condition.
- It is a fundamental requirement in algorithm design to guarantee reliability, accuracy, and stability in real-life systems and applications.

Definition:

- *Correctness of an algorithm means the algorithm must return the correct output for all possible valid inputs under defined constraints.*
- This concept ensures that the logic behind the algorithm is not just syntactically right but also functionally valid in practice.

Need for Algorithm Correctness:

- Without correctness, an algorithm may yield wrong results even if it compiles and runs without errors.
- Incorrect algorithms may lead to failures in applications, especially in critical areas like banking, healthcare, and aviation systems.
- Validating correctness helps developers identify logical errors early and build confidence in algorithm implementation.
- It prevents unexpected system behaviors, crashes, or data loss caused by incorrect logic or faulty control flow.
- Ensuring correctness is a key step before optimizing algorithms for performance or deploying them into production systems.

Definition: Loop Invariant Property:

- *A loop invariant is a condition that holds true before and after each iteration of a loop during the algorithm's execution.*

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.1

- Loop invariants are used to prove that an algorithm behaves correctly at every step and maintains required properties.

Proof of Correctness using Loop Invariant – Summation of n Numbers:

- Consider an algorithm to calculate the sum of first n natural numbers.

Algorithm:

```
sum = 0;
for (i = 1; i ≤ n; i++) {
   sum = sum + i;
}
```

Loop Invariant:

- At the start of each iteration of the loop, the variable sum contains the sum of all integers from 1 to i – 1.

Proof by Loop Invariant has three steps:

- *Initialization:*
  - Before the first iteration, i = 1, and sum = 0.
  - Since sum of numbers from 1 to 0 is 0, the invariant holds.
- *Maintenance:*
  - Assuming the invariant is true for i = k, so sum = 1 + 2 + ... + (k – 1).
  - In next iteration, sum = sum + k, which becomes 1 + 2 + ... + (k – 1) + k.
  - Hence, the invariant holds for i = k + 1.
- *Termination:*
  - When loop ends, i = n + 1, so the loop has added all values from 1 to n.
  - Final value of sum is 1 + 2 + ... + n, which is correct.

	Q.No.									TOTAL
E										
M										

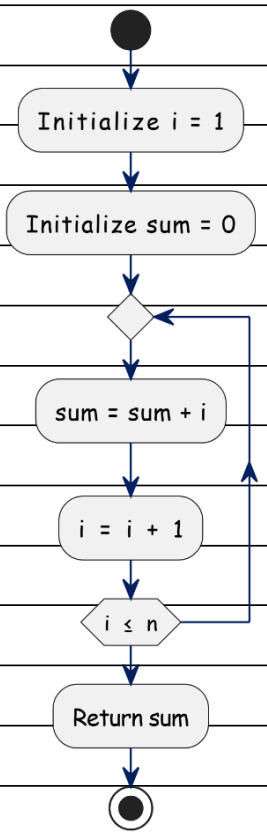
प्र.क्र./Q.No.

Q.1

diagram:

- figure : Loop Invariant for Sum of First n Numbers

figure : - Loop Invariant for Sum of First n Numbers



Explanation of diagram:

- The diagram describes the flow of summing numbers from 1 to n using a for loop with incrementing index i.
- The loop maintains the invariant that sum stores the total of all values from 1 to i–1 at the start of each cycle.
- After each addition and increment, the property still holds true, validating correctness by logical consistency.

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.1

-> b) (Iterative Algorithm Design Issues)

Introduction:

- Iterative algorithms use repetition to solve problems by repeatedly executing a block of statements using loops like for, while, or do-while.
- These algorithms continue execution until a certain condition is met, making them suitable for problems with known steps or bounds.

Definition:

- *Iterative algorithm:*
  - An algorithm that repeats a specific block of instructions until a condition is satisfied is called an iterative algorithm.
  - Loop structures like for-loop, while-loop, or do-while-loop are used to implement such logic.
- *Design issue:*
  - Design issues refer to common challenges or concerns that must be addressed for correct and efficient algorithm behavior.

Key Characteristics of Iterative Algorithms:

- They are deterministic and repeat operations over a fixed or dynamically changing loop condition.
- Efficiency, termination, and correctness must be carefully handled during their design.

Common Design Issues in Iterative Algorithms:

- *Loop Termination:*
  - The algorithm must ensure the loop ends after a finite number of steps to prevent infinite loops.
  - If not handled, it can lead to a program crash or memory overflow.
- *Correct Loop Condition:*

प्र.क्र./Q.No.

Q.1

- The condition controlling the loop must be correct to avoid skipping important operations or repeating unnecessarily.
- Misjudged conditions can lead to incorrect results or incomplete execution.

- *Variable Initialization and Update:*
  - All loop variables must be properly initialized and updated in each iteration.
  - Improper initialization can cause logical errors and produce unexpected outputs.
- *Memory or Space Complexity:*
  - Repeated operations may require large memory if not optimized.
  - Iterative algorithms must minimize space usage while retaining clarity.
- *Handling of Edge Cases:*
  - Iterative loops must consider cases like empty arrays, boundary values, or zero input to avoid crashes.
  - For example, iterating through an empty list without checks can lead to errors.
- *Correctness of Loop Invariant:*
  - A loop invariant is a condition that remains true during every iteration.
  - Ensuring the invariant helps to verify the correctness of logic inside the loop.

Examples of Iterative Algorithms with Design Observations:

- *Example 1: Finding sum of first n natural numbers*

```
int sum = 0;
for (int i = 1; i <= n; i++) {
   sum = sum + i;
}
```

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.1

- Here, i is correctly initialized and incremented.
- The loop terminates at i = n, avoiding overflow.
- Proper use of loop invariant: sum always holds the correct sum of numbers 1 to i.
- *Example 2: Searching an element in an array*

```
int found = 0;
for (int i = 0; i < n; i++) {
   if (A[i] == x) {
      found = 1;
      break;
   }
}
```

- Checks if the element x exists in array A using a loop.
- Ensures that loop exits as soon as element is found (efficient design).
- Addresses design issue of unnecessary looping after result is achieved.

Additional Related Notes:

- Iterative algorithms are generally easier to debug and understand than recursive ones.
- However, failure to manage loop conditions, initialization, and edge cases can result in design flaws.
- Choosing the correct loop construct (for, while) depends on the nature of the problem and known inputs.

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.2

-> a) (Proving Algorithm Correctness)

Introduction:

- Proving the correctness of an algorithm is essential to ensure that it works accurately under all possible valid inputs and conditions.
- It involves logically verifying that the algorithm produces the expected result by following well-defined steps or testing its failure through counter examples.

Definition:

- *Correctness of an algorithm means that for every input within the problem domain, the algorithm terminates and gives a correct output.*
- *Proof techniques such as mathematical induction, loop invariants, and counter examples are used to establish or question correctness.*

Methods to Prove Algorithm Correctness:

- Proving correctness involves two key properties:
  - *Partial correctness* – the algorithm gives the correct result if it terminates.
  - *Termination* – the algorithm always finishes after a finite number of steps.
- Common methods used for correctness proof include:
  - Using *loop invariants* to track and prove correctness at each iteration.
  - Using *mathematical induction* to prove recursive or iterative behavior.
  - *Counter examples* to show specific cases where algorithm fails, thus proving it incorrect.

Definition: Counter Example in Algorithm Correctness:

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.2

- *A counter example is a specific input or condition for which the algorithm fails to produce the correct output, proving it is not universally correct.*
- It is used to disprove an algorithm by finding even a single failure case.

Example: Incorrect Algorithm using Counter Example:

- Consider an algorithm that checks if an integer is even using the logic below:

```
bool isEven(int x) {
   return (x / 2) * 2 == x;
}
```

- Let's test it with a negative input x = –4.

Expected result: True, since –4 is even.

Output from algorithm:

- (–4 / 2) = –2, and (–2 * 2) = –4, which gives –4 == –4 → True.
- Now test with x = –3.

Expected result: False, since –3 is odd.

Output from algorithm:

- (–3 / 2) = –1 (integer division), and (–1 * 2) = –2,
  so –2 == –3 → False (Correct output).
- Now modify the function as follows:

```
bool isEven(int x) {
   return x % 2 == 0;
}
```

- Use counter example for an incorrect version of algorithm like:

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.2

```
bool isEven(int x) {
    return (x & 1) == 1;  // Wrong logic if expecting even check
}
```

- For x = 4, (x & 1) = 0 → so result is False.
  But this logic wrongly detects even numbers as odd.

Conclusion from counter example:

- The above algorithm fails for all even inputs and hence is incorrect.
- Just one counter example is enough to invalidate an algorithm's correctness completely.

-> b) (Problem Solving Strategies)

Introduction:

- In computer science, a *problem solving strategy* is a general method or technique used to design an algorithm that efficiently solves a given problem.
- Different strategies help to structure solutions and reduce the time and space complexity in various problem domains.

Definition:

- *Problem solving strategy:*
  - It is a systematic approach used to analyze the problem and design an optimal or correct algorithmic solution.

Important Problem Solving Strategies:

- *Divide and Conquer:*
  - This strategy divides the problem into smaller subproblems, solves each recursively, and combines their solutions to solve the original problem.

प्र.क्र./Q.No.

Q.2

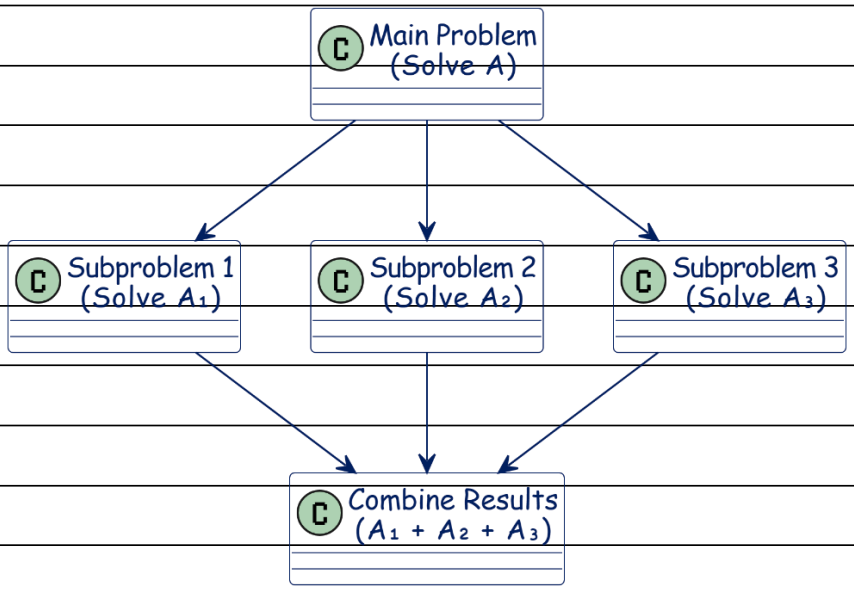
- Example: Merge Sort, Binary Search, Quick Sort.

- *Greedy Method:*
  - It makes the locally optimal choice at each step with the hope of finding the global optimum.
  - Example: Kruskal's Algorithm, Prim's Algorithm, Activity Selection Problem.

diagram:

- figure : Divide and Conquer Strategy

figure : - Divide and Conquer Strategy



Explanation of diagram:

- The diagram shows how a large problem is divided into smaller ones, which are then solved and merged to get the final result.
- This method enhances performance in problems where smaller solutions are easier to compute.
- *Dynamic Programming:*
  - This method solves each subproblem once and stores its result to avoid redundant computations (overlapping subproblems).

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.2

- Example: Fibonacci Series, Longest Common Subsequence, Knapsack Problem.

- *Backtracking:*
  - Backtracking is used when problems have multiple solution paths, and we need to go back when a path fails to reach a solution.
  - Example: N-Queens Problem, Sudoku Solver, Subset Sum Problem.
- *Brute Force:*
  - It tries all possible solutions to find the correct one. This is simple but not always efficient.
  - Example: Checking all passwords in a login cracker, generating all combinations for small input sizes.

Additional Examples of Usage:

- *Divide and Conquer in Binary Search:*
  - Array is repeatedly divided in half until the desired value is found or all elements are checked.
- *Greedy in Job Scheduling:*
  - Select job with earliest finish time first to schedule maximum tasks.
- *Dynamic Programming in Matrix Chain Multiplication:*
  - Stores minimum cost for sub-chains and uses those to solve larger chain problems.
- *Backtracking in N-Queens:*
  - Places queens row by row, backtracks when two queens threaten each other.

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.3

-> a) (Input Size, Best Case, Worst Case, Average Case)

Introduction:

- Algorithm analysis helps us predict the resources required (time or memory) as input size increases.
- Different scenarios arise depending on the input arrangement. These are called best case, worst case, and average case.

Definition:

- *Best Case:*
  - The minimum time an algorithm takes to run for any input. It occurs when the input is optimally arranged.
- *Worst Case:*
  - The maximum time an algorithm takes to run. It happens when the input is arranged in the least favorable order.
- *Average Case:*
  - The expected running time over all possible inputs. It gives a practical estimate of performance.

Explanation of Input Size:

- Input size (n) refers to the total number of elements passed to the algorithm.
- The complexity is measured in terms of how many times loops or operations execute relative to n.

Given Algorithm:

```
void sort (int a[ ], int n) {
   int i, j;
   for (i = 0; i < n; i++) {
```

प्र.क्र./Q.No.

Q.3

```
        j = i - 1;
        key = a[i];
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j = j - 1;
        }
        a[j + 1] = key;
    }
}
```

- The given algorithm is an *Insertion Sort* algorithm that sorts elements in ascending order.

Best Case Analysis:

- Happens when the input array is already sorted in increasing order.
- The while condition fails immediately every time: a[j] > key becomes false.
- Number of comparisons = (n - 1) and zero swaps.
- Hence, time complexity is linear – $O(n)$.

Worst Case Analysis:

- Happens when the input array is sorted in reverse (descending) order.
- Every element needs to be compared and shifted till it reaches the start.
- Number of comparisons = $1 + 2 + \ldots + (n - 1) = n(n - 1)/2$.
- Time complexity is quadratic – $O(n^2)$.

Average Case Analysis:

- Occurs when elements are in random order.
- In average case, for each i^n element, about $i/2$ comparisons are needed.
- So, total comparisons $\approx n^2/4$.
- Hence, average time complexity = $O(n^2)$.

Summary of Complexities:

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.3

- *Best Case:* Input is already sorted → $O(n)$
- *Worst Case:* Input is reverse sorted → $O(n^2)$
- *Average Case:* Input is random → $O(n^2)$

-> b.i) P, NP, NP-Hard and NP-Complete

Introduction:

- In complexity theory, problems are classified into different classes based on the resources required to solve them, especially time.
- The classes P, NP, NP-Hard, and NP-Complete help in understanding which problems can be solved or verified efficiently.

Definition of P Class Problems:

- P class refers to all decision problems that can be solved in polynomial time using deterministic algorithms.
- These are the easiest problems that can be both solved and verified quickly.
- Example: Checking if a number is prime or not using the AKS algorithm belongs to class P.

Definition of NP Class Problems:

- NP stands for Non-deterministic Polynomial time where solutions cannot be found quickly but can be verified in polynomial time.
- It is unknown whether NP problems can be solved in polynomial time like P problems.
- Example: The Subset Sum Problem where we check if a subset with a given sum exists.

Definition of NP-Complete Problems:

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.3

- NP-Complete problems are a subset of NP problems that are as hard as any problem in NP.
- If any NP-Complete problem is solved in polynomial time, all NP problems can also be solved in polynomial time.
- These problems are both in NP and NP-Hard.
- Example: The Boolean Satisfiability Problem (SAT) is an NP-Complete problem.

Definition of NP-Hard Problems:

- NP-Hard problems are at least as hard as NP problems but not necessarily in NP.
- These problems may not even be decision problems and may not have polynomial-time verification.
- Example: The Halting Problem is NP-Hard but not in NP because it is not verifiable.

Diagram:

- Diagram showing relationship between classes P, NP, NP-Complete, and NP-Hard
- figure : Class Relation Diagram

Examples of Problems in Each Class:

- *P*: Linear Search, Binary Search, Matrix Multiplication
- *NP*: Travelling Salesman Decision Problem, Subset Sum
- *NP-Complete*: 3-SAT, Vertex Cover, Clique Problem
- *NP-Hard*: Halting Problem, General Optimization Versions of NP-Complete Problems

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.3

-> b.ii) *NP-Complete Problems : 3-SAT:*

Introduction:

- In theoretical computer science, SAT and 3-SAT problems are central to understanding NP-complete problems in computational complexity theory.
- These problems involve determining the satisfiability of Boolean expressions, a core challenge in logic, computation, and algorithm analysis.

Definition:

- *SAT (Satisfiability Problem):*
  - SAT is the problem of determining if there exists an assignment of truth values to variables such that a Boolean formula becomes true.
- *3-SAT Problem:*
  - 3-SAT is a special case of SAT where the Boolean formula is in conjunctive normal form (CNF), and each clause has exactly three literals.

Explanation of SAT and 3-SAT with Example:

- A Boolean formula is composed of variables, logical AND ($\wedge$), OR ($\vee$), and NOT ($\neg$) operators.
- Example of SAT formula: $(x_1 \vee \neg x_2) \wedge (x_2 \vee x_3 \vee x_4)$
- In 3-SAT, each clause must have three literals. Example: $(x_1 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_3 \vee x_4)$
- To satisfy this 3-SAT formula, we must assign values (true/false) to x_1, x_2, x_3, x_4 so that the entire expression becomes true.

Importance of SAT in Theoretical Computer Science:

- SAT was the first problem proved to be NP-complete by Stephen Cook in 1971 (Cook's theorem).

प्र.क्र./Q.No.

Q.3

- The significance lies in its ability to express a wide range of decision problems as satisfiability problems.
- Solving SAT efficiently can help solve other NP problems through polynomial-time reductions.
- Many real-world applications, such as hardware verification, software testing, and artificial intelligence use SAT solvers.

Proof that 3-SAT is NP-Complete:

- *Step 1: 3-SAT is in NP class:*
  - Given a Boolean assignment, we can verify if a 3-SAT formula is satisfied in polynomial time.
- *Step 2: Reduction from SAT to 3-SAT:*
  - Every instance of SAT can be transformed into an equivalent 3-SAT formula using polynomial time reduction.
  - This proves 3-SAT is at least as hard as SAT.
- *Conclusion from both steps:*
  - Since SAT is NP-complete and 3-SAT is polynomially reducible from SAT, and is also in NP, hence 3-SAT is NP-complete.

Example to Prove NP-Completeness:

- Consider the formula: $(x_1 \vee x_2 \vee x_3) \wedge (x_4 \vee \neg x_5 \vee x_6)$
- Any SAT instance can be rewritten in the above form without changing the solution status.
- Thus, the satisfiability of general Boolean formulas can be decided using a 3-SAT solver, showing 3-SAT's NP-completeness.

Additional Note on Real-World Importance:

- SAT and 3-SAT problems are foundational in studying advanced complexity classes like NP, NP-complete, and NP-hard.
- They also serve as benchmarks for evaluating the efficiency of various heuristic and approximation algorithms.

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.4

-> a) *NP-Class of Problems*

Introduction:

- In complexity theory, problems are classified into different classes based on the resources required to solve them, especially time.
- The classes P, NP, NP-Hard, and NP-Complete help in understanding which problems can be solved or verified efficiently.

Definition of P Class Problems:

- P class refers to all decision problems that can be solved in polynomial time using deterministic algorithms.
- These are the easiest problems that can be both solved and verified quickly.
- Example: Checking if a number is prime or not using the AKS algorithm belongs to class P.

Definition of NP Class Problems:

- NP stands for Non-deterministic Polynomial time where solutions cannot be found quickly but can be verified in polynomial time.
- It is unknown whether NP problems can be solved in polynomial time like P problems.
- Example: The Subset Sum Problem where we check if a subset with a given sum exists.

Definition of NP-Complete Problems:

- NP-Complete problems are a subset of NP problems that are as hard as any problem in NP.
- If any NP-Complete problem is solved in polynomial time, all NP problems can also be solved in polynomial time.
- These problems are both in NP and NP-Hard.

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.4

- Example: The Boolean Satisfiability Problem (SAT) is an NP-Complete problem.

Definition of NP-Hard Problems:

- NP-Hard problems are at least as hard as NP problems but not necessarily in NP.
- These problems may not even be decision problems and may not have polynomial-time verification.
- Example: The Halting Problem is NP-Hard but not in NP because it is not verifiable.

Vertex Cover Problem and NP-Completeness:

- A vertex cover of a graph is a set of vertices such that every edge is incident on at least one vertex in the set.
- The problem of checking if a vertex cover of size k exists is known as the Vertex Cover Problem.
- This problem is in NP because if we are given a solution (vertex set), we can verify its correctness in polynomial time.

Proof of NP-Completeness of Vertex Cover:

- The vertex cover problem is proved NP-Complete by polynomial-time reduction from the SAT problem or the 3-SAT problem.
- This means that SAT problem is converted to vertex cover in polynomial time.
- Since SAT is NP-Complete and vertex cover is in NP, by reduction the vertex cover is also NP-Complete.
- Hence, solving the vertex cover problem efficiently would solve all NP problems efficiently.

Examples of Problems in Each Class:

- *P*: Linear Search, Binary Search, Matrix Multiplication

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.4

- *NP*: Travelling Salesman Decision Problem, Subset Sum
- *NP-Complete*: 3-SAT, Vertex Cover, Clique Problem
- *NP-Hard*: Halting Problem, General Optimization Versions of NP-Complete Problems

-> b) *Best, Average and Worst Case Analysis of Linear Search Algorithm*

Introduction:

- The performance of an algorithm depends on the input it receives and may vary for different cases of the same input size.
- The three main types of performance analysis are: best case, average case, and worst case analysis.
- These cases help evaluate algorithm efficiency under various conditions and guide in selecting suitable algorithms.

Definition:

- *Best Case:* The minimum number of steps taken by an algorithm for any input size.
- *Average Case:* The expected number of steps considering all possible inputs, assuming they occur with equal probability.
- *Worst Case:* The maximum number of steps taken by an algorithm for any input size.

Linear Search Algorithm Working:

- Linear search scans each element of the array one by one from index 0 to n-1.
- If the element is found, the function returns the element.
- If not found, it returns -1 indicating the item is not present in the array.

Best Case Analysis:

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.4

- Best case occurs when the item is found at the first position in the array.
- In such a case, the loop executes only once and returns immediately.
- *Time complexity:* O(1) — constant time.

Worst Case Analysis:

- Worst case occurs when the item is not present in the array or is at the last position.
- In this situation, the loop iterates through all n elements.
- *Time complexity:* O(n) — linear time.

Average Case Analysis:

- Average case assumes the item is equally likely to be at any position in the array.
- On average, the item will be found after scanning n divided by 2 elements.
- *Time complexity:* O(n) — linear time.

Diagram:

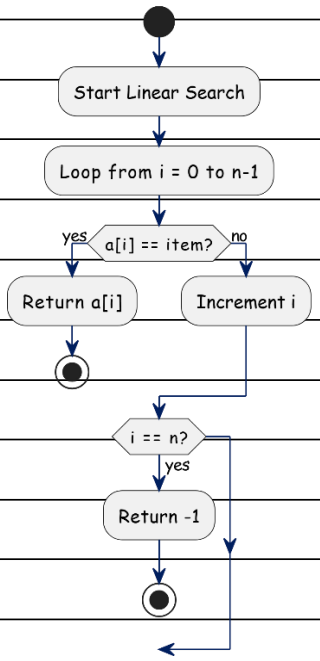


figure : Linear search control flow diagram

	Q.No.									TOTAL
E										
M										

प्र.क्र./Q.No.

Q.4

Explanation of Diagram:

- The diagram shows a simple decision path where a loop runs through each element.
- If match is found early, it exits quickly (best case).
- If no match is found until end, it returns -1 (worst case).
- The same path is partially followed for average case depending on where the match is found.

Observation:

- Best case performance of Linear Search is very fast with only one comparison.
- However, in practical scenarios where element may not be at the start, average or worst case dominates.
- Hence, for large datasets, more efficient search methods like binary search are preferred if data is sorted.